

Applications of Reinforcement Learning to Emulator-based Stateful Greybox Fuzzing

Master's thesis

Bastian Engel

December 16, 2025

1 Background

1.1 Fuzzing

Fuzz testing, or *fuzzing* for short, is a widely deployed method for automatic software testing and vulnerability discovery that makes use of a diverse set of tools.[1] Its goal is to find inputs to a Program Under Test (PUT) that trigger a certain behavior, often a crash that might then lead to the discovery of bugs or security vulnerabilities. One way to classify fuzzers is by the information that is available to them.

Fuzzers that only make use of externally observable behavior of the PUT are called *black-box* fuzzers. In this case, there might only be access to a crash oracle and no further conclusions about the behavior of the PUT under a certain input can be drawn. In contrast to this, *white-box* fuzzers allow full access to the PUT, usually including any original source code or documentation which can greatly aid the fuzzing process.

Since detailed information is often not available but observing certain parts of the internal behavior of the PUT might still be possible, *grey-box* fuzzers represent an important middle-ground between the previous two extremes. A common source of instrumentation data that grey-box fuzzers make use of is the *coverage map* of the PUT during execution. It allows the fuzzer to direct its exploration efforts towards promising regions of the PUT.

One famous example of a coverage-guided greybox fuzzer is AFL++[2]. It works by iteratively improving the *corpus*, a set of promising inputs, through a series of mutations and selections. Mutations include operations like bit flips, substitutions and splicing. Newly generated inputs are then evaluated and added to the corpus for further mutation and selection if they produce new coverage. The success of a greybox fuzzer is strongly dependent on a high-quality initial corpus and appropriate seed mutation and selection strategies.

1.2 Reinforcement learning

Reinforcement learning (RL) is an extremely general framework for approaching problems in which an agent strives to maximize a reward signal through a selection of actions in certain states.[3] These kinds of problems can be modeled as finite Markov decision processes (MDPs). A Markov decision process is a 3-tuple $MDP = (S, A, p)$ where

- S is the set of states,
- A is the set of actions where $\mathcal{A}_s \subseteq A$ are actions available in state $s \in S$,

- $p(s', r|s, a) = \Pr\{S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a\}$ describes the system dynamics, i.e., the probability of encountering a next state s' and reward r at time step t when taking action a in state s at time step $t - 1$

An RL agent's goal is to learn a policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ mapping each state s to probabilities of taking each possible action a to maximize the cumulative reward. Algorithms for finding an optimal π^* can be divided into three classes: value-based, policy-based and actor-critic.

Value-based methods try to learn a value function $Q : \mathcal{S} \rightarrow \mathcal{R}$ mapping states to their expected cumulative reward so that in each step, the action that maximizes the expected reward can be taken. These include well-known methods such as Q-learning, TD(λ) or SARSA.

Policy-based methods try to directly construct an optimal policy π^* without the use of a value function. They have various advantages. For example, they can easily handle continuous action spaces and ensure stochasticity which is important for state space exploration. A well-known classical policy-based algorithm is REINFORCE.[4]

Actor-critic methods synthesize value- and policy-based methods by including information about a state's value in the policy learning process.

Since classical, tabular methods don't work well with large state spaces, state-of-the-art methods make use modern-day deep learning advancements. Examples include the family of Deep-Q Network (DQN) algorithms[5][6] approximating the Q function by neural networks, policy-based gradient-descent methods like Proximal Policy Optimization (PPO)[7] and actor-critic methods like A3C[8]. Impressive results range from the master-level Go program AlphaGo[9] and its more generalized successors AlphaZero[10] and MuZero[11] to Dreamerv3[12] that is able to learn how to acquire diamonds in the popular video game *Minecraft* without any human input or expert knowledge.

2 Reinforcement learning for fuzzing

Fuzzing can be modeled as a reinforcement learning problem yadayaya...

2.1 Prior work

cite stuff, explain things

References

- [1] Valentin J.M. Manès et al. “The Art, Science, and Engineering of Fuzzing: A Survey”. In: *IEEE Transactions on Software Engineering* 47.11 (2021), pp. 2312–2331. DOI: [10.1109/TSE.2019.2946563](https://doi.org/10.1109/TSE.2019.2946563).
- [2] Andrea Fioraldi et al. “{AFL++}: Combining incremental steps of fuzzing research”. In: *14th USENIX workshop on offensive technologies (WOOT 20)*. 2020.
- [3] Richard S Sutton, Andrew G Barto, et al. “Reinforcement learning: An introduction”. Vol. 1. 1. MIT press Cambridge, 1998.
- [4] Ronald J Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine learning* 8.3 (1992), pp. 229–256.
- [5] Volodymyr Mnih et al. “Playing atari with deep reinforcement learning”. In: *arXiv preprint arXiv:1312.5602* (2013).
- [6] Matteo Hessel et al. “Rainbow: Combining improvements in deep reinforcement learning”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.
- [7] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [8] Volodymyr Mnih et al. “Asynchronous methods for deep reinforcement learning”. In: *International conference on machine learning*. PmLR. 2016, pp. 1928–1937.
- [9] David Silver et al. “Mastering the game of Go with deep neural networks and tree search”. In: *nature* 529.7587 (2016), pp. 484–489.
- [10] David Silver et al. “Mastering chess and shogi by self-play with a general reinforcement learning algorithm”. In: *arXiv preprint arXiv:1712.01815* (2017).
- [11] Julian Schrittwieser et al. “Mastering atari, go, chess and shogi by planning with a learned model”. In: *Nature* 588.7839 (2020), pp. 604–609.
- [12] Danijar Hafner et al. “Mastering diverse domains through world models”. In: *arXiv preprint arXiv:2301.04104* (2023).